

Agent-Mediated Knowledge Graph Chat — MVP Architecture

Purpose: A crisp, developer-ready blueprint for a web-based, agent-orchestrated, graph-based chat workspace where each node is an isolated chat + context, child nodes handle corrections, and a Router Agent dynamically injects relevant context into a single global LLM instance.

1. High-level overview

This system models a project as a directed graph of **Nodes** (atomic chat/context workspaces) and **Edges** (relationships: `depends_on`, `related_to`, `conflicts_with`). A **Main Node** aggregates outputs from dependent nodes. An orchestrator (Router Agent) composes context for the global LLM using per-node embedding indices and metadata. Child nodes are used for corrections or continuations and are linked to parents with full version history.

2. Core entities

Node

- `id`, `name`, `type` (`main`|`component`|`child`|`continuation`), `status` (`fresh`|`stale`|`merge_pending`), `owner` (`single-user`), `created_at`, `updated_at`.
- `history`: chronological chat messages and LLM responses.
- `vector_index`: per-node embeddings for semantic retrieval.
- `version_log`: commit-like snapshots.
- `hallucination_log`: recorded hallucination events with category labels.

Edge

- `source`, `target`, `relation_type` (`depends_on`|`related_to`|`conflicts_with`), `metadata` (`strength`, `notes`).
-

3. Actors / Agents

- **Router Agent:** Selects nodes and fragments to build the LLM context package. Applies relevance ranking, staleness checks, and decides when to create clarification/continuation nodes.
- **Merge Agent:** Runs semantic diffing, generates human-readable summaries of changes, enforces merge policy (sequential merges for MVP), and applies snapshots.

- **Clarification Agent:** Detects knowledge gaps and can auto-create question nodes or prompt the user.
-

4. Storage & Indexing

- **Graph DB:** Neo4j or ArangoDB — stores nodes, edges, metadata, version references.
 - **Vector Store(s):** FAISS (local), Weaviate, or Pinecone for per-node embedding indices.
 - **Version Storage:** Git-like snapshot storage (GitPython or custom) storing commit metadata + diffs.
 - **Optional Cache:** In-memory LRU cache for hot node contexts during active sessions.
-

5. Context & Token strategy

- Each node has a **2,000,000 token** allocation for stored context.
 - When overflow occurs, create a `Continuation` node linked to the parent and mark retrieval chains.
 - Use per-node embeddings to fetch *relevant fragments* (top-k) rather than whole history for each LLM call.
 - Implement **Node Staleness Flag**: when a node is updated and it has dependents, mark dependents `stale: true`.
-

6. Correction & Merge flow (user-driven)

Merger flow (summary):

1. User clicks **Edit** on hallucinated output.
2. System creates a `Child Node` (type=correction) pre-loaded with the snippet and surrounding context.
3. LLM-guided correction session occurs inside the child node.
4. User triggers **Request Merge**.
5. Merge Agent computes:
6. Text diff
7. Semantic summary (natural-language summary of intent change)
8. Conflict detection against other pending merges
9. Present user with a **Diff UI** (semantic + textual) — user approves or rejects.
10. On approval:
11. Create a version snapshot of parent
12. Apply changes
13. Propagate `stale` flags to dependents

Merge policy (MVP): Sequential merges. If multiple children modify same region, present them in queue order and create a merge-conflict child if needed.

7. Dynamic context composition (Router Agent logic)

- Input: active node + user query.
- Steps:
 - Parse the query to extract explicit node references (e.g., `#Node_DB`) and implicit intent.
 - Use graph traversal (Neo4j/ArangoDB) to collect candidate nodes (dependencies, related nodes, recent edits).
 - For each candidate node, run semantic search on its vector store to fetch top-N relevant fragments.
 - Apply staleness filter: prefer `fresh` nodes; if a dependent is `stale`, include its `hallucination_log` and prompt the user to revalidate.
 - Compose context package: prioritized fragments + short metadata header (node names, statuses, latest commit hashes).
- Send to global LLM instance.

8. Main Node aggregation modes

- **Live mode (default):** Always reads latest versions of dependent nodes at runtime.
- **Snapshot mode:** Uses explicit version references (commits) for each dependency; safe for reproducible builds.

Provide a UI toggle on Main Node for the chosen mode.

9. UI/UX

- **Canvas:** tldraw/Excalidraw-like canvas. Right-click context menu: `Create Node`, `Ask Question`, `Add Dependency`.
- **Node panel:** Click node → opens a tabbed chat panel on the right.
- **Sidebar:** Node list + filters (Recent, Stale, Merge Pending). Mini-map for global navigation.
- **Inline references:** `#Node_X` becomes hoverable with preview and link.
- **Status badges:** Fresh / Stale / Merge Pending.
- **Diff Viewer:** Split view (semantic summary on left, textual diff on right) with approve/reject buttons.

10. Tech stack (MVP)

Frontend

- React + tldraw/Excalidraw

- Monaco editor for code diffs/edits
- Zustand for client state

Backend

- FastAPI (Python)
- Neo4j or ArangoDB
- FAISS (local) or Weaviate
- GitPython (snapshots)
- LangChain-like orchestration + custom Router/Merge agents

LLM

- Single global instance (OpenAI/Anthropic/Local LLM). Keep abstraction layer for swap-in.

11. Example API endpoints (brief)

- `POST /nodes` → create node
- `GET /nodes/{id}` → fetch node metadata + preview
- `POST /nodes/{id}/messages` → append message
- `POST /router/query` → router composes context and queries LLM
- `POST /nodes/{parent_id}/child` → create child/correction node
- `POST /merge/{child_id}` → request merge
- `GET /nodes/{id}/diff/{commit_a}/{commit_b}` → fetch textual/semantic diff

12. Node lifecycle (states)

- `draft` → initial node created
- `active` → being edited or queried
- `merge_pending` → child changes awaiting merge
- `stale` → dependent updated, needs revalidation
- `archived` → old continuation or superseded nodes

13. Hallucination & knowledge-gap metrics

- Track: `hallucination_count`, `hallucination_types` (MissingFact, WrongAssumption, Speculation), `last_detected_at`.
- Produce dashboard: per-node heatmap of hallucination frequency and per-topic gap suggestions.

14. Security & single-user constraints

- Single-user mode simplifies permissions: no real-time collaboration controls initially.
 - Store encryption-at-rest for user data; session tokens for LLM access.
-

15. Roadmap (v0 → v1)

v0 (MVP)

- Web UI canvas + clickable nodes
- Neo4j + FAISS per-node vector store
- Global LLM + Router Agent
- Child node creation & manual merge flow
- Version snapshots & diff viewer

v1

- Staleness automation & background revalidation jobs
 - Merge-conflict resolution UI (semi-automated)
 - Knowledge-gap dashboard & suggestions
 - Local LLM support & hybrid caching
-

16. Next steps (developer-ready)

1. Scaffold repo: frontend (React) + backend (FastAPI). Add sample seed data for nodes/edges.
 2. Implement Node CRUD + basic Neo4j schema.
 3. Wire simple vector store per node (FAISS) and index seed messages.
 4. Build Router Agent prototype that composes context (simple top-k fragments) and calls a stubbed LLM.
 5. Implement child node creation + merge flow with textual diffs.
 6. Replace LLM stub with real provider and iterate UX.
-

17. Appendix — Mermaid flow diagrams

```
flowchart LR
  User((User)) -->|query| MainNode[Main Node]
  MainNode --> Router[Router Agent]
  Router --> Neo4jDB[Graph DB]
  Router --> VectorStore[Per-node Vector Stores]
```

```
Router --> LLM[Global LLM]
LLM --> MainNode
MainNode -->|mark hallucination| ChildNode[Child Node]
ChildNode --> MergeAgent[Merge Agent]
MergeAgent --> MainNode
MergeAgent --> Neo4jDB
```

End of document.